Lidox Platform

Assignment 1: Requirements Engineering, Architecture & Proof of Concept

Collaborative Document Editor with AI Writing Assistant

VERSION
3.0 — Final

DATE
April 2026

COURSE
AI1220

TEAM
Lidox Engineering

COMPLIANCE
ISO 27001, ISO 25010
RFC 6455, 6749, 7519, 8446

DIAGRAMS
draw.io / Mermaid
(source files attached)

Contents

Part 1: Requirements Engineering

1 Stakeholder Analysis

Stakeholder	Goals / Concerns	Influence on Requirements
Product Owner / Startup Founder	*Goal:* Deliver a competitive AI-powered editor with fast time-to-market and predictable unit economics. *Concern:* Runaway LLM API costs; feature delays; weak user adoption.	Drives modular, API-first design; task-aware LLM budgeting; cost circuit breakers; iterative delivery.
End Users (Writers, Students, Teams)	*Goal:* Frictionless real-time collaboration, intuitive AI that preserves their voice. *Concern:* Data loss during edits, laggy sync, confusing AI rewrites, format corruption.	Requires CRDT-based conflict resolution, $\leq$150 ms sync latency, AI-as-proposal UX, offline resilience.
Engineering Team (Developers)	*Goal:* Maintainable, testable codebase with clear service boundaries. *Concern:* Tight coupling between real-time and CRUD; hard-to-test AI features.	Leads to separation of concerns, API-first contracts, mockable AI orchestration layer.
DevOps / SRE	*Goal:* High availability, horizontal scalability, full observability. *Concern:* WebSocket connection management at scale; database write storms from ephemeral presence data.	Requires tiered state architecture (Redis for ephemeral, PostgreSQL for durable), isolated real-time infrastructure.
AI/LLM Provider (External Service)	*Goal:* Stable, efficient API consumption within rate limits. *Concern:* Burst traffic, oversized prompts, retry storms.	Forces asynchronous AI processing with queue-backed execution, exponential backoff, token metering.
Enterprise Customers / Org Admins	*Goal:* Data security, compliance (ISO 27001), role-based access control, auditability. *Concern:* Data leakage to third-party LLMs, unauthorized access, ghost permission windows.	Requires strong authentication with active revocation, document-level AI kill switch, configurable retention.

2 Functional Requirements

Format Convention

Each requirement specifies: unique ID, description, triggering condition, expected system behavior, and acceptance criteria—per the assignment rubric.

2.1 Area 1: Real-Time Collaboration

High-level capability: Multiple users simultaneously edit the same document with sub-second state synchronization, live presence awareness, and deterministic conflict resolution without data loss.

FR-1.1 State Synchronization (CRDT)

The system shall synchronize document state across concurrent clients using a Conflict-free Replicated Data Type (CRDT) transmitted via persistent WebSocket connections (RFC 6455).

Trigger: A user types, deletes, or formats text in a shared document.

Expected Behavior: The CRDT engine generates a local operation, applies it optimistically to the local editor state, and broadcasts the operation to all connected peers via the Real-Time Service. All replicas converge to an identical document state without coordination.

Acceptance Criteria: Given 2+ concurrent editors making simultaneous edits to the same paragraph, the final document state is identical across all clients within 300 ms (p99) and no characters are lost.

FR-1.2 Presence Broadcasting

The system shall display real-time presence indicators (cursor position, active selection, user name, avatar color) for all connected collaborators.

Trigger: A user moves their cursor or changes their text selection.

Expected Behavior: The client sends a lightweight presence update to the Real-Time Service via WebSocket. The service fans out the update to all other connected clients of that document via Redis Pub/Sub. Presence data is ephemeral (never written to the primary database).

Acceptance Criteria: When User A moves their cursor, User B sees the updated cursor position within 200 ms. When User A disconnects, their cursor disappears within 60 s (TTL expiry).

FR-1.3 Offline Reconciliation

The client shall buffer local edits during network partitions and merge them deterministically upon reconnection.

Trigger: The client detects a WebSocket disconnection (network loss, server restart).

Expected Behavior: Local CRDT operations are buffered in the browser's IndexedDB. Upon reconnection, the client sends buffered operations to the server and receives any missed remote operations. The CRDT merge algorithm resolves all conflicts deterministically without user intervention. If offline for $>$24 h, the client receives a full compacted snapshot instead of incremental operations.

Acceptance Criteria: A user edits 3 paragraphs while offline for 10 minutes. Upon reconnection, all offline edits appear in the document within 5 s and no data from either the offline or online users is lost.

2.2 Area 2: AI Writing Assistant

High-level capability: Users invoke contextual AI operations (rewrite, summarize, translate, restructure) on selected text. AI suggestions are presented as reviewable proposals anchored to CRDT state, with full format preservation and task-aware cost management.

FR-2.1 AI Invocation

The user can invoke an AI operation on selected text via a contextual menu.

Trigger: The user selects text and chooses an AI action (rewrite, summarize, translate, restructure, grammar fix) from the toolbar or keyboard shortcut.

Expected Behavior: The frontend captures the selected text, the CRDT node ID, and the current state vector. It sends the request to the Backend API, which routes it to the AI Orchestration Service. The AI service constructs a prompt using a task-aware token budget, enqueues the job, and returns an immediate acknowledgment with a task ID. The client shows a pending/loading state.

Acceptance Criteria: The AI pending indicator appears within 300 ms of invocation. The first token of the streamed response appears within 2 s for selections under 1,000 words.

FR-2.2 AI Proposal Display (Diff UX)

AI-generated text shall render as a reviewable diff proposal, not as an automatic replacement.

Trigger: The AI Orchestration Service completes processing and returns the generated text.

Expected Behavior: The client renders the AI output as a tracked-change-style overlay: additions highlighted in green, deletions in red. The user can accept all, reject all, accept individual sentences, or manually edit the proposed text before committing. Proposals are anchored to CRDT node IDs (not character offsets) and include a conflict resolution mechanism: if the source text has been modified by a collaborator during AI processing, the proposal is marked "stale" and the user is prompted to regenerate.

Acceptance Criteria: (1) The user can accept/reject an AI proposal. (2) Partial acceptance (individual sentences) works correctly. (3) A proposal generated against text that was concurrently edited by another user displays a "stale" indicator rather than silently applying.

FR-2.3 Rich-Text Preservation

The AI pipeline shall preserve formatting (bold, italic, links, headings, lists, tables) through the prompt–response round trip.

Trigger: The user invokes an AI operation on formatted text.

Expected Behavior: The ProseMirror-to-LLM serializer converts document nodes to a compact structured format with inline semantic tags. The AI response parser reconstructs ProseMirror nodes from the output, rejecting any output that violates the document schema. Tables are serialized as CSV-like representations; code blocks are passed verbatim.

Acceptance Criteria: A paragraph containing bold text, a hyperlink, and an inline code span, when rewritten by the AI, retains all three formatting marks in the output.

FR-2.4 AI Restructure / Outline

The user can ask the AI to restructure a document outline or reorder sections.

Trigger: The user selects multiple headings/sections and invokes "Restructure."

Expected Behavior: The AI receives the full heading tree and section summaries (heading-bounded chunking). It returns a proposed new outline with reordered/renamed sections. The proposal shows a before/after outline diff. Acceptance rearranges the actual document nodes.

Acceptance Criteria: Given a 5-section document, the AI produces a restructured outline. The user can preview the new order and accept or reject it. Upon acceptance, all content moves to the correct positions.

2.3 Area 3: Document Management

High-level capability: Full document lifecycle management including creation, versioning, sharing with role-based permissions, and export to standard formats.

FR-3.1 Document Creation

Users can create new documents with optional templates.

Trigger: The user clicks "New Document" from the dashboard.

Expected Behavior: The system creates a new document record in PostgreSQL, initializes an empty CRDT state in Redis, assigns the creating user as Owner, and redirects to the editor view. The new document is immediately editable.

Acceptance Criteria: A newly created document appears in the user's dashboard within 1 s and is editable by the creator immediately.

FR-3.2 Version History & Revert

The system shall maintain document version history and allow authorized users to revert to any previous version.

Trigger: The user opens the version history panel or clicks "Revert" on a specific version.

Expected Behavior: The system persists snapshots at configurable intervals (every 100 CRDT operations, every 5 minutes, or on explicit save). The version history panel shows timestamped snapshots with author attribution. Reverting replaces the current CRDT state with the snapshot state and broadcasts the change to all connected clients.

Acceptance Criteria: (1) At least 10 historical versions are available for a document edited over 1 hour. (2) Reverting to a version from 30 minutes ago replaces the content for all connected users within 3 s.

FR-3.3 Document Sharing & Access Control

Users can share documents with specific users or via link, with role-based permissions.

Trigger: The document owner opens the sharing panel and enters an email address or generates a share link with a specific role.

Expected Behavior: The system creates a permission record linking the target user/link to the document with one of four roles: Owner, Editor, Commenter, or Viewer. Permissions are enforced on every API request and WebSocket message. Commenter role allows AI "Analyze" but disables "Apply Changes." Viewer role disables all editing and AI invocation.

Acceptance Criteria: (1) A user shared as "Viewer" cannot type in the editor. (2) A "Commenter" can invoke AI analysis but the "Apply" button is disabled. (3) Permission changes take effect within 60 s for active sessions.

FR-3.4 Document Export

The system shall export documents to PDF and DOCX formats.

Trigger: The user clicks "Export" and selects a format.

Expected Behavior: The backend renders the current document state to the requested format. Unaccepted AI suggestions are rendered as margin annotations (tracked changes in DOCX, comment boxes in PDF). The exported file is returned as a downloadable binary.

Acceptance Criteria: An exported DOCX file opens correctly in Microsoft Word with formatting intact. Unresolved AI proposals appear as tracked changes.

2.4 Area 4: User Management

High-level capability: Secure authentication with federated SSO, short-lived tokens with active revocation, and role-based authorization enforced across all system boundaries.

FR-4.1 Authentication (Local + SSO)

The system shall support email/password authentication and SSO via Google and GitHub.

Trigger: The user navigates to the login page and enters credentials or clicks an SSO button.

Expected Behavior: Local credentials are validated against bcrypt-hashed passwords (cost factor $\geq$12). SSO follows the OAuth 2.0 Authorization Code flow (RFC 6749) with PKCE. Upon successful authentication, the server issues an access JWT (15-min TTL, HttpOnly Secure cookie) and an opaque refresh token (7-day TTL, stored server-side). The user is redirected to their dashboard.

Acceptance Criteria: (1) A user can log in with email/password. (2) A user can log in with Google SSO. (3) The JWT cookie has `HttpOnly`, `Secure`, and `SameSite=Strict` flags.

FR-4.2 Session Management & Token Revocation

The system shall enforce short-lived sessions with active revocation capability.

Trigger: An admin revokes a user's access, or a user logs out, or a refresh token is reused (replay attack detected).

Expected Behavior: The revoked JWT's `jti` is added to a Redis deny set (`SISMEMBER` check on every request). A Pub/Sub revocation event forces all Real-Time Service nodes to terminate the affected WebSocket connections within 5 s. Refresh token rotation invalidates old tokens; reuse of an old token triggers revocation of the entire token family.

Acceptance Criteria: (1) After revocation, the user's API requests return 401 within 15 minutes (JWT expiry). (2) The user's WebSocket connection is terminated within 60 s (re-auth ping cycle). (3) Refresh token reuse triggers immediate session termination.

FR-4.3 Role Enforcement

The system shall enforce four document-level roles: Owner, Editor, Commenter, Viewer.

Trigger: Any API request or WebSocket message that modifies document state or invokes AI.

Expected Behavior: The Backend API and Real-Time Service check the requesting user's role against the required permission level for the action. Unauthorized actions return HTTP 403 or a WebSocket error frame. The AI Orchestration Service additionally checks whether the org admin has disabled AI for the document.

Acceptance Criteria: (1) A Viewer attempting to send a CRDT operation receives a 403 error. (2) A Commenter invoking "AI Rewrite" receives a 403; invoking "AI Analyze" succeeds. (3) An Org Admin disabling AI for a document blocks all AI invocations for all roles.

3 Non-Functional Requirements

3.1 Latency

- **NFR-01** Keystroke propagation to connected peers: p50 $\leq$80 ms, p95 $\leq$150 ms, p99 $\leq$300 ms. *Justification:* Research shows collaborative presence breaks at $>$200 ms; $\leq$150 ms (p95) maintains the psychological illusion of instantaneous shared editing.
- **NFR-02** AI pending indicator: $\leq$300 ms. First token streaming: $\leq$2 s for tasks under 1,000 words. *Justification:* $>$300 ms without feedback causes users to re-click; $>$3 s feels "broken."
- **NFR-03** Document load (up to 100 pages, broadband): $\leq$1.5 s to interactive. *Justification:* Google's research shows 53% of mobile users abandon sites that take $>$3 s to load.

3.2 Scalability

- **NFR-04** 50 concurrent active editors per document without violating NFR-01.
- **NFR-05** 10,000 active documents and 100,000 authenticated users system-wide. *Growth model:* Initial target supports classroom and small-enterprise scenarios. Horizontal scaling via Redis Cluster partitioning (by document ID) and stateless backend pods.

3.3 Availability

- **NFR-06** Core editing platform: 99.9% monthly uptime (8.76 h/year max downtime).
- **NFR-07** If the AI subsystem is unavailable, users can still create, edit, save, share, and export documents (graceful degradation). The AI toolbar shows a "temporarily unavailable" indicator.
- **NFR-08** During temporary real-time service failure, clients buffer edits locally in IndexedDB and auto-sync on reconnection. Users see a "reconnecting" indicator but are never locked out of their document.

3.4 Security & Privacy

- **NFR-09** All data in transit: TLS 1.3 (RFC 8446). All data at rest: AES-256 (ISO 27001 compliance).
- **NFR-10** JWT access tokens: 15-min TTL. Active revocation latency: $\leq$60 s for WebSocket, $\leq$15 min for REST.
- **NFR-11** AI context sends minimum required data per task type. Raw prompt/response retained $\leq$30 days (org-configurable to 0). Document data never sent to LLM if org admin disables AI. *Privacy implication:* Sending content to third-party LLM APIs means the LLM provider may process/log that content. Org admins must be able to opt out entirely for sensitive documents.

3.5 Usability

- **NFR-12** When $>$10 collaborators are active, the presence UI collapses inactive cursors into a summary indicator ("+8 others") to prevent clutter.
- **NFR-13** AI suggestions are always previewable proposals that can be accepted, rejected, or edited before insertion. The user is never surprised by automatic content changes.
- **NFR-14** WCAG 2.1 Level AA compliance. The editor is keyboard-navigable. Screen readers announce AI

proposal state changes (“AI suggestion available,” “suggestion accepted”).

4 User Stories and Scenarios

Story Format

Each story includes expected system behavior and a design justification for non-obvious choices, per the rubric requirement.

Collaboration Scenarios

US-01 Presence Awareness

As a collaborator, I want to see labeled cursors of other active users so I avoid overwriting their work.

Expected Behavior: When a user opens a shared document, the Real-Time Service sends the current presence state of all connected users. Each collaborator’s cursor is rendered with their display name and a unique color. Cursor positions update at up to 10 Hz (server-throttled). When a user disconnects, their cursor disappears after the 60 s TTL expires.

Design Justification: Presence is broadcast via Redis Pub/Sub independently of document state to avoid write amplification on the primary database. Throttling to 10 Hz prevents fan-out amplification in documents with 50 editors.

US-02 Offline Editing & Reconnection

As a commuter, I want to edit during a tunnel outage and have my changes seamlessly merge when I reconnect.

Expected Behavior: The client detects disconnection and shows a “Working offline” banner. All local edits continue normally against the local CRDT state and are buffered in IndexedDB. Upon reconnection, buffered operations are sent to the server and remote operations are received. The CRDT merge algorithm resolves all conflicts deterministically. The “offline” banner disappears.

Design Justification: We use CRDTs (not OT) specifically because they support offline-first merging without server round-trips. The 24-hour snapshot fallback prevents unbounded operation replay for long-offline clients.

US-03 Concurrent Same-Paragraph Editing

As a co-author, I want to edit a paragraph simultaneously with a colleague without locking.

Expected Behavior: Both users’ keystrokes are applied locally via optimistic UI and broadcast via CRDT operations. The CRDT algorithm (Yjs/YATA) ensures character-level interleaving without paragraph-level locks. Neither user experiences any blocking or conflict dialog.

Design Justification: Paragraph-level locking was rejected (ADR-02) because it destroys the collaborative experience. CRDTs allow true character-level concurrency at the cost of higher implementation complexity.

US-04 Version Revert During Active Editing

As a team lead, I want to revert to a version from 2 hours ago, forcing a live refresh for all users.

Expected Behavior: The user opens the version history panel, selects a snapshot, and clicks “Revert.” The server replaces the current CRDT state with the snapshot state, broadcasts the new state to all connected clients via WebSocket, and each client replaces its local editor state. All cursors reset to position 0. A version entry is created recording who reverted and when.

Design Justification: A forced revert is disruptive by design—it is an emergency action. We broadcast the full replacement state rather than trying to compute a delta, because reverting to an old version may invalidate hundreds of intermediate CRDT operations.

AI Assistant Workflows

US-05 AI Summarization

As a writer, I want to select a long section and ask the AI for a bulleted summary.

Expected Behavior: The user selects text spanning multiple paragraphs and clicks “Summarize” from the AI toolbar. The system shows a loading indicator within 300 ms. The AI Orchestration Service constructs a prompt using heading-bounded context (up to 8,000 tokens). The response streams back and renders as a diff proposal showing the original text and the proposed summary. The user can accept, reject, or edit before insertion.

Design Justification: Summarization requires more context than grammar fixes. The task-aware token budget (8,000 for summarize vs. 800 for grammar) prevents hallucination from insufficient context while avoiding unnecessary cost for simple operations.

US-06 Partial AI Acceptance

As an editor, I want to accept only specific sentences from an AI rewrite while rejecting others.

Expected Behavior: The diff proposal renders each sentence as an independently selectable unit. The user clicks checkboxes next to individual sentences to include or exclude them. Clicking “Apply Selected” commits only the checked sentences to the CRDT state. Unchecked sentences are discarded.

Design Justification: Partial acceptance is critical for maintaining authorial voice. The alternative—all-or-nothing acceptance—forces users to manually re-edit AI output, negating the productivity benefit.

US-07 AI Translation with Format Preservation

As a researcher, I want to translate a section while preserving bolding and hyperlinks.

Expected Behavior: The user selects formatted text and invokes “Translate to [language].” The structured serializer converts ProseMirror nodes to tagged format, preserving inline markers. The AI receives the tagged text with a system prompt instructing format preservation. The response parser reconstructs ProseMirror nodes with original formatting intact.

Design Justification: We serialize to a compact tagged format (not Markdown) because Markdown introduces round-trip ambiguity (e.g., ∗ vs. _ for italics). Tagged format tokenizes more predictably and preserves exact formatting markers.

US-08 AI Suggestion Becomes Stale During Collaboration

As a user, I request an AI rewrite. While waiting for the result, my colleague edits the same paragraph. The AI suggestion arrives but is now based on outdated text.

Expected Behavior: The system detects that the source text’s SHA-256 hash no longer matches the current node content. Instead of silently applying the stale proposal, the system shows it with a yellow “Stale: text has changed” banner and a “Regenerate” button. The user can choose to regenerate with updated text or dismiss the proposal.

Design Justification: Silently applying stale proposals would corrupt content—a trust-destroying failure. We anchor proposals to CRDT state vectors (ADR-06) rather than character offsets specifically to detect and handle this race condition.

Document Lifecycle

US-09 Share with Read-Only Access

As a team lead, I share a document with a client as “Viewer.” They can read but not edit, invoke AI, or comment.

Expected Behavior: The owner enters the client’s email in the sharing panel and selects “Viewer.” The system creates a permission record. When the client opens the document, the editor renders in read-only mode: the cursor is non-interactive, the toolbar is disabled, and the AI menu is hidden.

Design Justification: We enforce permissions at both the UI level (disabled controls) and the API level (403 on write attempts) for defense-in-depth. UI-only enforcement is insufficient because a technical user could send raw API requests.

US-10 Export with AI Annotations

As a project manager, I export a document. Unresolved AI suggestions appear as tracked changes.

Expected Behavior: The user clicks Export → DOCX. The backend iterates over any pending AI proposals and converts them to DOCX tracked-change markup (using `w:ins` and `w:del` elements). The resulting file opens in Microsoft Word with suggestions visible in the review pane.

Design Justification: Discarding unresolved suggestions on export would lose work. Rendering them as tracked changes leverages an existing UX pattern that document reviewers already understand.

Access Control & Roles

US-11 Commenter Invokes AI Analyze (Allowed) vs. AI Rewrite (Blocked)

As a commenter, I can use AI to analyze text for my understanding, but I cannot apply changes to the document.

Expected Behavior: The user with "Commenter" role selects text and opens the AI menu. "Analyze" and "Explain" options are available; "Rewrite," "Summarize," and "Translate" show a lock icon with tooltip "Requires Editor role." If the commenter invokes "Analyze," the AI response displays in a side panel (read-only) with no "Apply to Document" button.

Design Justification: Commenters need AI for comprehension (analyzing complex text) but must not modify shared content. The role matrix differentiates between read-side AI (analysis) and write-side AI (rewrite/summarize). This granularity was requested by enterprise stakeholders.

US-12 Org Admin Disables AI for Confidential Documents

As an Org Admin, I disable AI features for a specific document to prevent sensitive content from being sent to external LLM APIs.

Expected Behavior: The admin opens the document settings panel and toggles "AI Features" to off. The system sets an `ai_enabled=false` flag on the document record. All AI invocations for this document are immediately blocked at the API layer (returning 403 with message "AI disabled by administrator"). The AI toolbar in the editor is hidden for all users.

Design Justification: Enterprise customers handling legal, medical, or financial data cannot risk any content reaching third-party LLM providers. This toggle provides document-level granularity rather than org-wide, allowing mixed environments.

US-13 AI Interaction History Review

As a professor, I want to review AI interaction history to understand how a section evolved and distinguish AI-generated from manually typed text.

Expected Behavior: The version history panel includes an "AI Activity" tab showing: timestamp, user who invoked AI, task type (rewrite/summarize/translate), source text snippet, and whether the suggestion was accepted, rejected, or partially applied. AI-contributed text is subtly marked in the editor with a small indicator icon. The audit log retains this data for the org-configurable retention period.

Design Justification: This supports academic integrity use cases and editorial review workflows. The retention policy balances auditability with GDPR Article 17 (right to erasure) obligations.

5 Requirements Traceability Matrix

Component Legend

C1: Frontend App C2: Backend API C3: Sync Server C4: AI Orchestration C5: Auth/Identity C6: PostgreSQL C7: Object Storage C8: Redis Cluster C9: Job Queue

Story	Func. Req.	Scenario	Components
US-01	FR-1.2	Presence cursors	C1, C3, C8
US-02	FR-1.1, FR-1.3	Offline editing & reconnection	C1, C3, C6, C8
US-03	FR-1.1	Concurrent same-paragraph editing	C1, C3, C8
US-04	FR-3.2, FR-1.1	Version revert during live session	C1, C2, C3, C6, C7
US-05	FR-2.1, FR-2.2	AI summarization with proposal UX	C1, C2, C4, C8, C9
US-06	FR-2.2	Partial AI acceptance	C1, C4

US-07	FR-2.1, FR-2.3	AI translation with formatting	C1, C2, C4, C9
US-08	FR-2.2, FR-2.4	Stale proposal / AI restructure	C1, C4, C8
US-09	FR-3.3, FR-4.3	Share as Viewer	C1, C2, C5, C6
US-10	FR-3.4	Export with AI annotations	C1, C2, C7
US-11	FR-4.3, FR-2.1	Commenter AI restrictions	C1, C2, C4, C5
US-12	FR-2.1, FR-4.3	Org Admin disables AI	C1, C2, C5, C6
US-13	FR-3.2, FR-2.1	AI interaction history audit	C1, C2, C6
Additional FR coverage (not mapped to a single story but covered by multiple):			
—	FR-3.1	Document creation	C1, C2, C6, C8 (via US-12, US-04)
—	FR-4.1	Authentication (local + SSO)	C1, C2, C5, C6 (via US-09, US-11, US-12)
—	FR-4.2	Session mgmt & token revocation	C2, C5, C8 (via US-11, US-12)

Part 2: System Architecture

6 Architectural Drivers

Why This Ranking Matters

Two teams with different driver rankings should arrive at different architectures. Our ranking reflects a product where *real-time collaboration reliability* is non-negotiable, *AI quality/cost* is the primary business risk, and *security* is the enterprise sales gate.

Rank 1: Real-Time Collaboration Latency & Reliability (NFR-01, NFR-06, NFR-08)

If sync is slow or lossy, the product is unusable regardless of AI quality. This driver forces: CRDT-based conflict resolution (ADR-02), a dedicated Real-Time Service separated from CRUD traffic (ADR-01), a tiered state architecture with Redis for ephemeral/warm data (ADR-05), and offline-first client design. Every infrastructure choice flows downstream from the $\leq$150 ms p95 latency target.

Rank 2: AI Integration Quality & Cost Control (NFR-02, NFR-11, FR-2.1–2.4)

The AI assistant is the product differentiator but also the primary cost and reliability risk. This driver forces: task-aware token budgeting with model tiering (ADR: implicit in FR-2.1), asynchronous queue-backed execution (ADR-04), anchor-based proposal conflict resolution (ADR-06), and per-org cost circuit breakers. Without these, a single viral document can generate thousands of dollars in LLM costs in hours.

Rank 3: Security & Compliance (NFR-09, NFR-10, FR-4.1–4.3)

Enterprise customers will not adopt a tool that fails security review. This driver forces: active JWT revocation with Redis deny set (ADR-07), document-level AI kill switch, TLS 1.3 everywhere, AES-256 at rest, and configurable data retention policies. The 60-second WebSocket revocation target specifically addresses ISO 27001 A.9.2.6.

Rank 4: Developer Velocity & Maintainability (Stakeholder: Engineering Team)

A startup must ship fast. This driver forces: monorepo with shared type definitions, API-first contracts between services, mockable AI orchestration layer, and clear code ownership boundaries. It also informs our choice of proven technologies (React, NestJS, Yjs) over novel but risky alternatives.

7 System Design Using the C4 Model

7.1 Level 1: System Context Diagram

The Lidox platform interacts with two actor types and four external services.

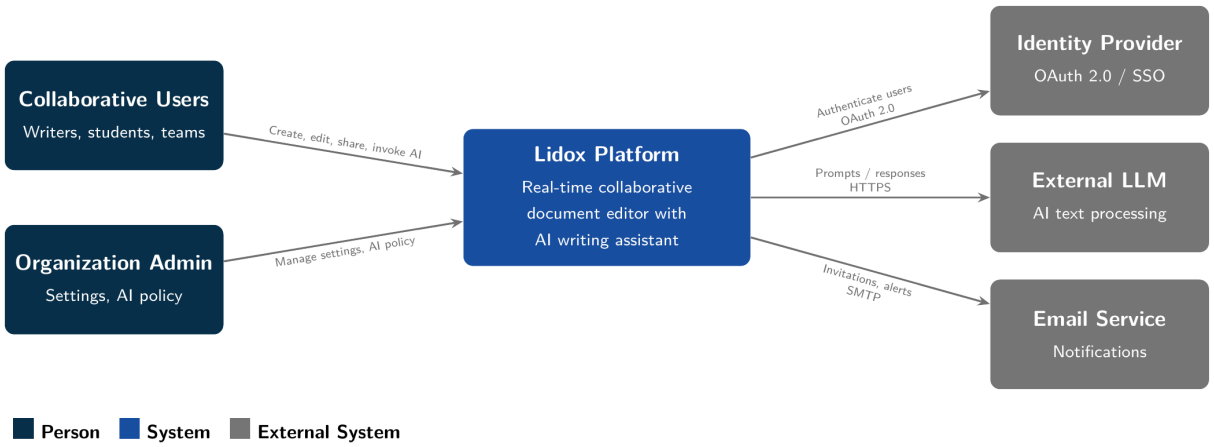

Figure 1: *
Level 1: System Context Diagram (Mermaid source: `diagrams/context.mmd`)

7.2 Level 2: Container Diagram

Architectural Invariant: Data Tiering

Ephemeral data (presence, cursors) flows **only** through Redis (Tier 1). Active CRDT state is cached in Redis with async write-behind to PostgreSQL (Tier 2). Durable data (users, permissions, audit) flows **only** through PostgreSQL (Tier 3). Violating this tiering re-introduces the write-amplification vulnerability identified in the architectural review.

Summary of containers:

Container	Responsibility	Technology	Protocol
C1: Frontend Web App	Editor UI, presence rendering, AI suggestion UX, offline buffering	React + TipTap (ProseMirror)	HTTPS, WebSocket
C2: Backend API	REST endpoints, document CRUD, permissions, AI routing	Node.js / NestJS	HTTPS REST
C3: Sync Server	WebSocket management, CRDT operation relay, presence fan-out	Node.js + Yjs + Hocuspocus	WebSocket
C4: AI Orchestration	Prompt construction, task-aware budgeting, provider adapter, format parsing	Node.js / Python	Internal HTTP
C5: Auth/Identity	JWT issuance, refresh rotation, deny set management	Passport.js + Redis	Internal
C6: PostgreSQL	Users, documents, permissions, audit logs, AI interaction history	PostgreSQL 16	TCP
C7: Object Storage	Version snapshots, exported documents	S3-compatible	HTTPS
C8: Redis Cluster	Presence (Tier 1), CRDT cache (Tier 2), deny set, Pub/Sub	Redis 7 Cluster	TCP
C9: Job Queue	Async AI task execution, retries with exponential backoff	BullMQ on Redis	Internal

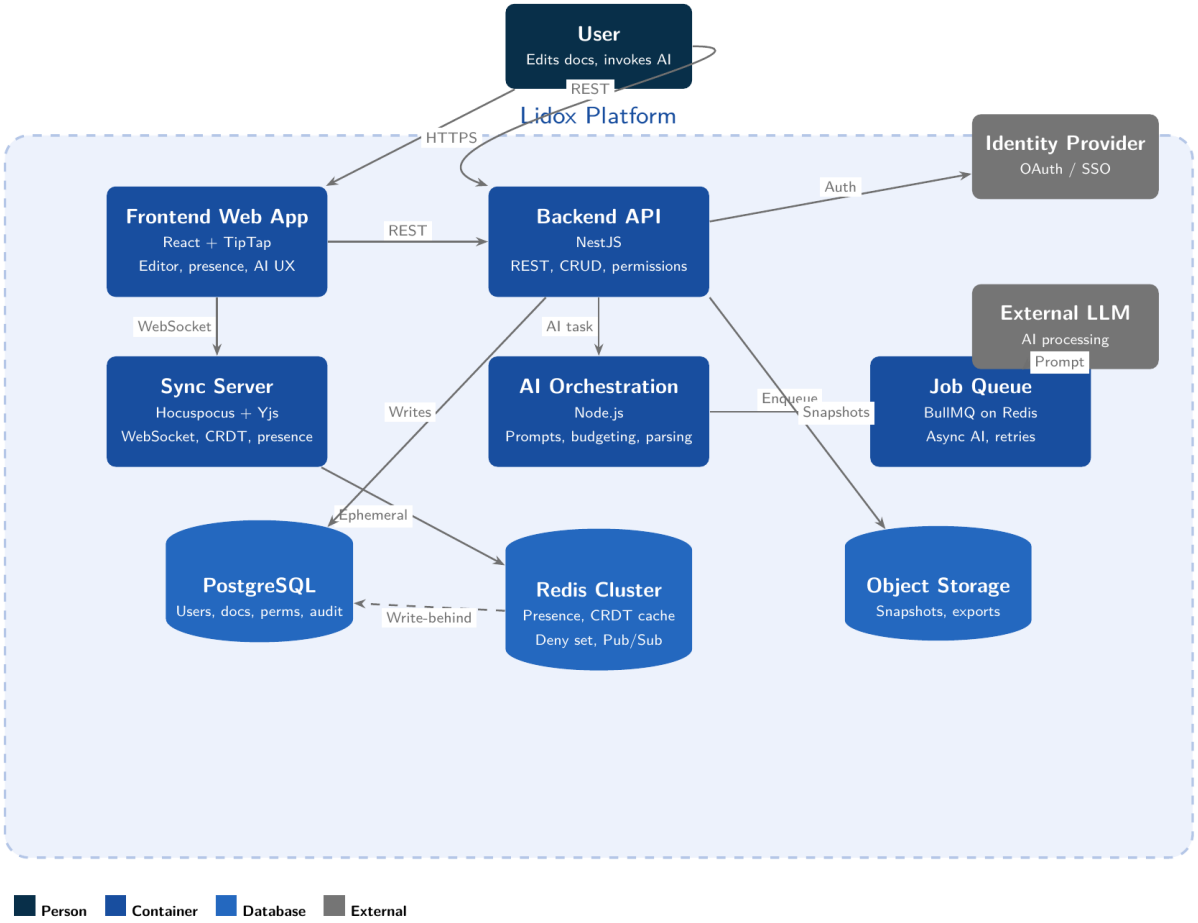

Figure 2: *
Level 2: Container Diagram (Mermaid source: `diagrams/container.mmd`)

7.3 Level 3: Component Diagram — AI Orchestration Service (C4)

Why AI Orchestration

We chose this container for Level 3 decomposition because it is the most architecturally complex: it touches frontend (proposal UX), backend (routing), database (audit), queue (async execution), and external services (LLM API).

Component	Responsibility	Depends On
Request Router	Receives AI invocation requests from C2. Validates user role and document AI-enabled flag. Routes to appropriate handler based on task type.	C2, C5, C6
Context Builder	Extracts selected text + surrounding context from the CRDT state. Applies task-aware token budget (800–16,000 tokens). Strips CRDT metadata and collaboration artifacts.	C8 (CRDT cache)
Prompt Template Engine	Selects and hydrates prompt templates based on task type. Templates are stored as configurable YAML files (hot-reloadable without redeployment). Supports system prompt, user prompt, and few-shot examples.	Config store
Structured Serializer	Converts ProseMirror node trees to compact tagged format for LLM consumption. Converts tables to CSV-like representation. Handles the reverse: parsing LLM output back to ProseMirror nodes, rejecting schema-violating output.	—
LLM Provider Adapter	Abstracts the LLM API call. Supports multiple providers (OpenAI, Anthropic, open-source). Routes tasks to appropriate model tier (fast/standard/premium). Handles streaming, retries (exponential backoff, 3 attempts), and timeout management.	External LLM

Token Metering & Cost Engine	Counts input/output tokens using provider tokenizers. Logs per-request metrics. Maintains per-org cost accumulator. Triggers alerts at 80% budget and hard-stop at 100%. Enforces per-request ceiling ($0.50 without confirmation).	C6, C8
Proposal Manager	Packages the LLM response with the original CRDT state vector and source text hash. Performs conflict detection (match/rebase/stale). Manages proposal TTL (120 s auto-dismiss).	C8
Audit Logger	Records AI interactions: user ID, document ID, task type, token counts, cost, accept/reject status. Respects retention policy ($\leq$30 days configurable).	C6

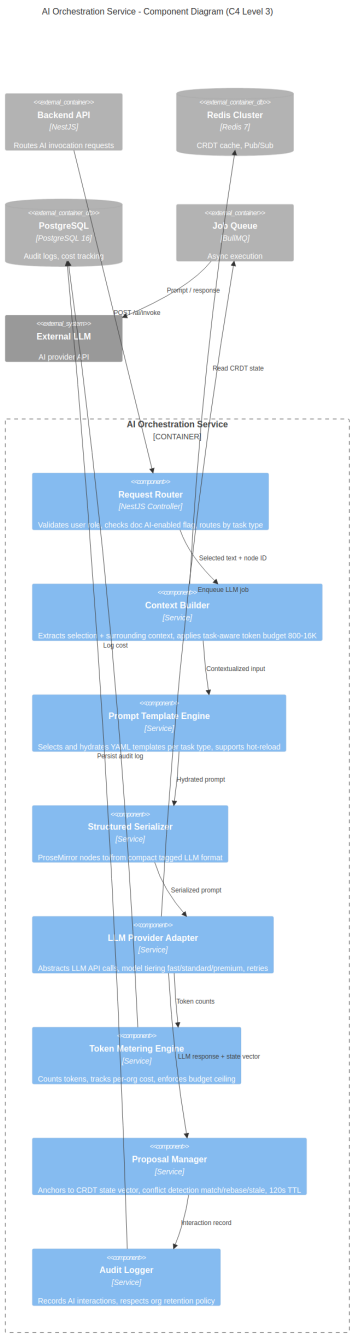

Figure 3: *
Level 3: AI Orchestration Service — Component Diagram (Mermaid source: `diagrams/component-ai.mmd`)

7.4 Feature Decomposition

Each module's responsibility, dependencies, and exposed interface:

Module	Responsibility	Dependencies	Interface
Rich-Text Editor (Frontend)	ProseMirror/TipTap editor rendering, toolbar actions, AI proposal overlay, offline IndexedDB buffer	TipTap, Yjs client, React state	React components; emits CRDT ops, AI invocation events
Real-Time Sync Layer (`sync-server`)	WebSocket connection management, CRDT operation relay, presence fan-out, reconnection handling, GC coordination	Yjs server (Hocuspocus), Redis Pub/Sub	WebSocket protocol; accepts/broadcasts CRDT ops and presence frames
AI Assistant Service	Prompt construction, LLM routing, response parsing, proposal lifecycle, cost metering (see Level 3 above)	Job Queue, LLM Provider, Redis, PostgreSQL	Internal REST API; accepts `POST /ai/invoke`, returns streaming events
Document Storage & Versioning	Document CRUD, snapshot persistence, version history queries, export rendering	PostgreSQL, Object Storage, Redis (CRDT cache)	REST API endpoints for document lifecycle
Auth & Authorization	Login/SSO flows, JWT issuance, refresh rotation, deny set management, role enforcement middleware	PostgreSQL (users, permissions), Redis (deny set), Identity Provider	Middleware; guards API routes and WebSocket connections
API Layer	REST endpoint routing, request validation (Zod schemas), error handling, rate limiting, OpenAPI documentation	NestJS framework, all service modules	HTTPS REST; OpenAPI spec at `/api/docs`

7.5 AI Integration Design

7.5.1 Context and Scope

The AI sees **only the minimum context required** for the task, never the full document by default. A "Grammar Fix" operation receives the selected text plus one sentence before/after (800 token ceiling). A "Summarize" operation receives the full heading-bounded section (8,000 tokens). A "Full Document Analysis" is the only task that sees the complete document (chunked at 16,000 tokens per request). This minimizes cost, latency, and data exposure to the LLM provider.

For very long documents ($>$16K tokens), the system uses **heading-based chunking**: it splits the document at heading boundaries and processes each chunk separately, assembling the results on the client.

7.5.2 Suggestion UX

AI suggestions are presented as **tracked-change-style proposals**: additions highlighted green, deletions highlighted red, rendered inline in the editor. The user can accept all, reject all, accept individual sentences, or manually edit the proposal text. A side panel shows a clean "before/after" view. Proposals auto-dismiss after 120 seconds of inactivity to prevent ghost suggestions. Accepted suggestions are committed to the CRDT state as a single atomic operation; undo reverts the entire acceptance.

7.5.3 AI During Collaboration

We do **not** lock the region during AI processing. Other users can continue editing freely. When the AI result arrives, the anchor-based conflict resolution system (FR-2.2) detects whether the source text has changed. If unchanged: apply directly. If surrounding text changed: rebase offsets and show a "Context Changed" indicator. If the source text itself changed: mark as stale, prompt "Regenerate?" This was chosen over region-locking because locking a paragraph for 2–8 seconds in a multi-user session is unacceptably disruptive.

7.5.4 Prompt Design

Prompts are **template-based**, stored as YAML files in the repository under `packages/ai-service/prompts/`. Each template has a system prompt (persona, constraints), a user prompt (dynamic: selected text, context), and optional few-shot examples. Templates are hot-reloadable: changing a YAML file and restarting the AI service updates prompts without a full application redeployment. Future iteration: configurable prompts per organization.

7.5.5 Model and Cost Strategy

We use **tiered model routing**: fast models (e.g., Claude Haiku) for grammar/spelling (high frequency, low complexity), standard models (e.g., Claude Sonnet) for rewrite/translate/summarize, and premium models (e.g., Claude Opus) for full-document analysis. This reduces LLM API costs by an estimated 40–60% versus a single-model approach. Cost management: per-org monthly budget with 80% alert and 100% hard-stop. Per-request ceiling of $0.50 requires explicit user confirmation. Token counts are metered per request and surfaced in an admin dashboard.

7.6 API Design

7.6.1 API Style

REST (JSON over HTTPS) for all CRUD operations and AI invocation. WebSocket (RFC 6455) for real-time CRDT sync and presence. AI long-running operations use a hybrid: the initial invocation is a REST `POST` that returns a task ID; results are streamed to the client via the existing WebSocket connection (event type: `ai:result`).

7.6.2 Endpoint Contracts

Method	Endpoint	Description	Auth
Document CRUD			
`POST`	`/api/documents`	Create document. Body: `{title, templateId?}`. Returns: `{id, title, createdAt, role}`	JWT
`GET`	`/api/documents/:id`	Get document metadata + initial CRDT state snapshot. Returns: `{id, title, ..., snapshot}`	JWT+Role≥Viewer
`PATCH`	`/api/documents/:id`	Update metadata (title, settings). Body: `{title?, aiEnabled?}`	JWT+Role≥Owner
`DELETE`	`/api/documents/:id`	Soft-delete document.	JWT+Role=Owner
Real-Time Session (WebSocket)			
`WS`	`/ws/documents/:id`	Upgrade to WebSocket. Auth via cookie JWT. Server sends initial state + presence. Client sends CRDT ops and presence updates.	JWT (cookie)
AI Assistant			
`POST`	`/api/documents/:id/ai/invoke`	Invoke AI. Body: `{task, selection, nodeId, stateVector}`. Returns: `{taskId, status:"queued"}`	JWT+Role≥Editor
`GET`	`/api/documents/:id/ai/tasks/:taskId`	Poll task status (fallback if WS disconnected). Returns: `{status, result?, error?}`	JWT
Sharing & Permissions			
`POST`	`/api/documents/:id/share`	Share document. Body: `{email, role}`. Returns: `{permissionId}`	JWT+Role≥Owner
`GET`	`/api/documents/:id/permissions`	List all permissions for a document.	JWT+Role≥Owner
`DELETE`	`/api/documents/:id/permissions/:pid`	Revoke a permission. Triggers deny-set update.	JWT+Role=Owner
User/Auth			
`POST`	`/api/auth/register`	Register. Body: `{email, password, name}`	None
`POST`	`/api/auth/login`	Login. Body: `{email, password}`. Sets JWT + refresh cookies.	None
`POST`	`/api/auth/refresh`	Rotate tokens. Uses refresh cookie.	Refresh token
`GET`	`/api/auth/sso/:provider`	Initiate OAuth flow (Google/GitHub).	None

7.6.3 Long-Running AI Operations

The client invokes AI via `POST /ai/invoke` and receives an immediate `202 Accepted` with a `taskId`. The AI result streams back through the WebSocket connection as `ai:chunk` events (for streaming) and an `ai:complete` event (final). If the WebSocket is disconnected, the client can poll `GET /ai/tasks/:taskId` as a fallback.

7.6.4 Error Handling Strategy

HTTP Code	Meaning	Client Behavior
202	AI task accepted, processing	Show loading indicator; wait for WebSocket events
400	Malformed request (validation error)	Show inline form validation errors
401	JWT expired or invalid	Silently attempt token refresh; if refresh fails, redirect to login
403	Insufficient role / AI disabled	Show “Permission denied” toast with reason
429	Rate limit or budget exceeded	Show “AI quota exceeded” banner with org admin contact
503	AI service unavailable	Show “AI temporarily unavailable” with retry option; editing continues
504	AI timeout (>30 s)	Show “AI request timed out” with retry option

7.7 Authentication & Authorization

Why authentication is needed: The platform handles potentially sensitive document content shared among specific users. Without authentication, we cannot enforce role-based access, audit AI usage, or manage

per-org billing.

User types: Individual users (personal accounts), team members (invited to shared workspaces), and organization administrators (manage settings, AI policies, billing).

Role matrix:

Action	Owner	Editor	Commenter	Viewer
View document	✓	✓	✓	✓
Edit document	✓	✓		
Add comments	✓	✓	✓	
Invoke AI (write: rewrite, summarize, translate)	✓	✓		
Invoke AI (read: analyze, explain)	✓	✓	✓	
Revert version	✓	✓		
Share / manage permissions	✓			
Delete document	✓			
Disable AI for document	Org Admin			

Privacy implications of LLM APIs: Sending content to third-party LLM providers means the provider may process, log, or temporarily store that content. Mitigations: (1) send minimum context per task; (2) org admins can disable AI per document; (3) AI interaction logs are retained for $\leq$30 days (configurable to 0); (4) system prompt instructs the LLM not to retain or learn from user content.

7.8 Communication Model

Model: Push-based real-time communication via persistent WebSocket connections. When a user edits the document, the CRDT operation is applied locally (optimistic UI) and sent to the Real-Time Service over WebSocket. The service relays the operation to all other connected clients. There is no polling.

Trade-offs: WebSockets provide the lowest-latency bidirectional communication (critical for our $\leq$150 ms p95 target) but require persistent connections, which are more complex to scale horizontally than stateless HTTP. We accept this complexity because polling-based sync (Server-Sent Events + HTTP POST) would add 200–500 ms of latency per keystroke, destroying the collaboration experience.

Document open: When a user opens a shared document, the client (1) fetches the initial document metadata via REST, (2) upgrades to a WebSocket connection, (3) receives the current CRDT state snapshot and presence state, (4) renders the document and begins accepting edits.

Disconnection and reconnection: On WebSocket disconnection, the client switches to offline mode: edits are applied locally and buffered in IndexedDB. A “reconnecting” banner is shown. On reconnection, buffered operations are replayed to the server, and missed remote operations are received. CRDT merge ensures consistency. If offline for $>$24 h, the client receives a full compacted snapshot.

8 Code Structure & Repository Organization

8.1 Monorepo Strategy

We use a **monorepo** (managed by Turborepo) for frontend, backend, shared types, and AI service. Justification for our team size ($\sim$4 developers): a monorepo enables atomic cross-cutting changes (e.g., an API contract change updates types, backend, and frontend in a single PR), eliminates version drift between packages, and simplifies CI/CD. Multi-repo would be appropriate for larger teams ($>$15) with independent deploy cadences.

8.2 Directory Layout

```
lidox/
  apps/
    web/                                    # React + TipTap frontend (Vite)
      public/                                          # Static assets
      src/                                  # Components, hooks, stores, lib
    api/                                               # NestJS backend API
      src/
        auth/                               # Login, SSO, JWT, refresh, guards
        documents/                                  # CRUD, versioning, export
        permissions/                               # Sharing, role enforcement
        ai/                                        # AI routing, orchestration
        middleware/                                 # Auth guard, rate limiter
    sync-server/                                 # Hocuspocus + Yjs WebSocket server
      src/                                # Extensions:  auth, persistence, presence
  packages/
    types/                                # Shared TypeScript interfaces, Zod schemas
  diagrams/                                        # Mermaid source files (.mmd)
  docs/                                               # ADRs, decisions, specs
  docker-compose.yml                                # Local dev (PostgreSQL, Redis)
  .env.example                                             # Template (no secrets)
  turbo.json                                          # Turborepo pipeline config
  README.md                                  # Setup guide, architecture overview
```

8.3 Shared Code

The `packages/types` package contains TypeScript interfaces and Zod validation schemas shared between frontend and backend. This ensures API request/response types are validated at compile time and runtime on both sides. Shared types are imported as a workspace dependency (`"@lidox/types": "workspace:*"`).

8.4 Configuration & Secrets Management

API keys, database connection strings, and LLM provider credentials are stored as environment variables, never committed to the repository. `.env.example` documents required variables with placeholder values. In production: secrets are injected via Kubernetes Secrets or a vault service (e.g., HashiCorp Vault). The `.gitignore` includes `.env*` (except `.env.example`).

8.5 Testing Structure

Tests are co-located with source code (`*.spec.ts` next to `*.ts`). Three test levels: **Unit tests** (Jest)—pure functions, serializer, prompt builder. **Integration tests** (Supertest)—API endpoint contracts validated against shared Zod schemas. **E2E tests** (Playwright)—frontend-to-backend flows (document creation, AI invocation). AI integration tests use **recorded fixtures** (saved LLM responses) to avoid real API calls and cost during CI.

9 Data Model

9.1 Entity-Relationship Overview

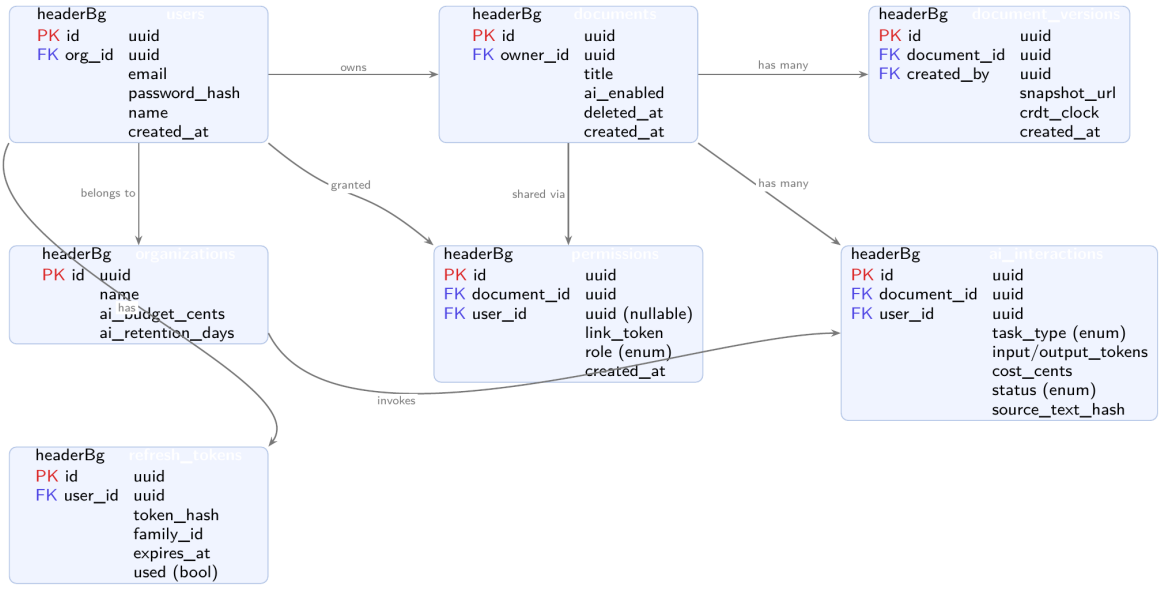

Figure 4: *
Entity-Relationship Diagram (Mermaid source: `diagrams/erd.mmd`)

9.2 Table Definitions

Table	Key Columns	Notes
`users`	`id` (UUID PK), `email` (unique), `password_hash`, `name`, `avatar_url`, `org_id` (FK), `created_at`	SSO users have null `password_hash`
`organizations`	`id` (UUID PK), `name`, `ai_budget_cents`, `ai_budget_used_cents`, `ai_retention_days`	Org-level AI settings
`documents`	`id` (UUID PK), `title`, `owner_id` (FK→users), `ai_enabled` (bool), `created_at`, `updated_at`, `deleted_at`	Soft-delete; `ai_enabled` for admin toggle
`document_versions`	`id` (UUID PK), `document_id` (FK), `snapshot_url` (S3 path), `crdt_clock`, `created_by` (FK), `created_at`	Snapshot stored in Object Storage
`permissions`	`id` (UUID PK), `document_id` (FK), `user_id` (FK, nullable), `link_token` (nullable), `role` (enum: owner/editor/commenter/viewer), `created_at`	Nullable `user_id` for link-based sharing
`ai_interactions`	`id` (UUID PK), `document_id` (FK), `user_id` (FK), `task_type` (enum), `input_tokens`, `output_tokens`, `model_used`, `cost_cents`, `status` (enum: accepted/rejected/partial/expired), `source_text_hash`, `created_at`	Prompt/response text retained per org policy
`refresh_tokens`	`id` (UUID PK), `user_id` (FK), `token_hash`, `family_id` (UUID), `expires_at`, `used` (bool)	`family_id` for rotation-based replay detection

9.3 Versioning Strategy

Documents are not stored as monolithic text blobs. The *live* document state is the CRDT state held in Redis (Tier 2), which is asynchronously persisted to PostgreSQL at $\leq$5 s intervals as a binary CRDT snapshot. Named *version snapshots* are triggered by: every 100 CRDT operations, every 5 minutes of active editing, or explicit user action. Snapshots are stored in Object Storage (S3) and referenced by URL in the `document_versions` table.

9.4 AI Interaction History

Every AI invocation creates a row in `ai_interactions` recording the full audit trail: who invoked what, on which document, with what cost, and whether the result was accepted. This supports the professor use case (US-13) and org admin auditing. Raw prompt/response text is stored in a separate `ai_interaction_details` table with TTL-based auto-deletion per the org's `ai_retention_days` setting.

10 Architecture Decision Records

ADR-01: Dedicated Real-Time Collaboration Service ACCEPTED

Context	Real-time editing has fundamentally different latency requirements than standard CRUD. WebSocket connections are long-lived and stateful; REST endpoints are short-lived. Mixing them in one service couples their scaling and failure modes.
Decision	Separate real-time synchronization into a dedicated service (C3) with its own scaling group. REST API (C2) is a separate service.
Consequences	+ Performance-critical traffic is isolated; each service scales independently. – Inter-service coordination adds complexity; document state must be accessible from both services.
Alternatives	*Single monolith*—simpler but latency-coupled. *Polling-based sync*—inferior UX, higher server load.

ADR-02: CRDT-Based Conflict Resolution (Yjs/YATA) ACCEPTED

Context	The platform requires simultaneous multi-user editing without locks. The offline reconciliation requirement (FR-1.3) demands merge without server round-trips.
Decision	Use Yjs (YATA algorithm) for its production maturity, compact binary encoding, TipTap integration, and native GC hooks.
Consequences	+ True concurrent editing without locks; offline-first by design. – Higher complexity; tombstone accumulation requires GC (NFR-07 defines triggers).
Alternatives	*OT*—requires central server for transformation, conflicts with offline-first. *Pessimistic locking*—destroys collaboration UX. *Last-write-wins*—causes data loss.

ADR-03: AI Outputs as Reviewable Proposals (Not Auto-Replace) ACCEPTED

Context	AI rewrites can be incorrect or tonally inappropriate. Auto-replacing content without consent is trust-destroying, especially in shared documents.
Decision	All AI outputs render as tracked-change-style diff proposals requiring explicit user action (accept/reject/partial/edit) before CRDT commit. Proposals auto-dismiss after 120 s.
Consequences	+ User maintains complete control; partial acceptance enables fine-grained editing. – Extra frontend complexity; users must take an additional action.
Alternatives	*Inline auto-replacement*—fast but risky in shared docs. *Full-document auto-rewrite*—destructive as default.

ADR-04: Async Queue-Backed AI Execution ACCEPTED

Context	LLM responses range from 2–15 s with unpredictable latency spikes and rate limits. Blocking threads on LLM calls would exhaust the connection pool.
Decision	Route AI requests through a durable job queue (BullMQ on Redis). Client receives immediate 202 with task ID; results stream via WebSocket. Retries use exponential backoff with jitter.
Consequences	+ Core app never blocks on LLM latency; clean retry semantics; quota enforcement at queue level. – More infrastructure; client handles async status updates.
Alternatives	*Synchronous backend-to-LLM*—fragile. *Direct client-to-LLM*—exposes API keys, bypasses audit.

11 Compliance & Standards Cross-Reference

Standard	Applicability	Addressed In
RFC 6455 (WebSocket)	Real-time CRDT transport, presence	FR-1.1, FR-1.2, C3
RFC 6749 (OAuth 2.0)	Federated auth (Google, GitHub SSO)	FR-4.1, C5
RFC 7519 (JWT)	Access token format and claims	FR-4.2
RFC 6819 (OAuth Threats)	Refresh token rotation, replay detection	FR-4.2
RFC 8446 (TLS 1.3)	Data-in-transit encryption	NFR-09

ISO/IEC 27001:2022	InfoSec management, access revocation (A.9.2.6), encryption at rest	NFR-09, NFR-10, FR-4.2
ISO/IEC 25010:2023	Software quality model	NFR-01 through NFR-14
WCAG 2.1 Level AA	Accessibility	NFR-14
GDPR Art. 17	Right to erasure for AI interaction data	NFR-11

12 Team Structure & Ownership

12.1 Code Ownership Areas

Each team member owns a primary area of the codebase. Ownership means: first point of contact for bugs, responsible for code review approvals in that area, and accountable for test coverage.

Ownership Area	Scope (Directories)	Key Responsibilities
Brandon — Architecture Lead	`docs/`, `diagrams/`, cross-cutting specs	System design, ADRs, C4 diagrams, requirements engineering, Mermaid diagrams, specification documents. Reviews all cross-cutting PRs.
Mohamed — Frontend & UX Lead	`apps/web/`	TipTap editor integration, AI suggestion overlay, presence rendering, offline IndexedDB buffer, state management, accessibility (WCAG 2.1 AA), responsive UI.
Abror — AI & Orchestration Lead	`apps/api/src/ai/`, `apps/sync-server/`	AI Orchestration Service, prompt templates, LLM provider adapter, token metering, Hocuspocus/Yjs sync server, CRDT operation relay, presence fan-out.
Abdelmonaim — Backend & DevOps Lead	`apps/api/` (core), `packages/types/`, infra	NestJS modules (auth, documents, permissions), REST API contracts, Zod schemas, role enforcement, Docker Compose, CI/CD (GitHub Actions), Redis/PostgreSQL setup.

12.2 Cross-Cutting Feature Handling

Features that span multiple ownership areas (e.g., the AI assistant, which touches frontend, backend API, AI service, and real-time layer) are handled as follows:

1. **Feature lead:** The team member whose area is most affected takes the lead. For the AI assistant, Abror leads because the AI Orchestration Service is the most complex component.
2. **Interface contracts first:** Before implementation begins, all involved owners agree on the API contract (request/response schemas in `packages/types`). This contract is merged as the first PR for the feature.
3. **Parallel implementation:** Each owner implements their side against the agreed contract. Integration tests validate the contract.
4. **Integration PR:** The feature lead opens a final PR that wires everything together and runs E2E tests.

12.3 Technical Disagreement Resolution

When team members disagree on a technical choice:

1. The two parties each write a brief (max 200-word) argument in the GitHub issue.
2. If unresolved in 24 hours, the team holds a 15-minute timeboxed discussion.
3. If still unresolved, the person whose ownership area is most affected makes the final call and documents the decision as a lightweight ADR in `docs/decisions/`.

13 Development Workflow

13.1 Branching Strategy

We use **GitHub Flow** (trunk-based with feature branches):

- **Main branch (`main`):** Always deployable. Protected: requires passing CI and at least 1 approved review.
- **Feature branches:** Named `feat/{issue-number}-short-description` (e.g., `feat/42-ai-proposal-overlay`).
- **Fix branches:** Named `fix/{issue-number}-short-description`.
- **Merge policy:** Squash merge to `main`. The squash commit message follows the Angular Conventional Commit format: `type(scope): description` (e.g., `feat(ai): add task-aware token budgeting`).
- **No long-lived branches:** Feature branches should be merged within 3 days. If a feature takes longer, break it into smaller PRs behind a feature flag.

13.2 Code Review Process

- **Every PR requires at least 1 approval** from a team member who is *not* the author.
- **Cross-area PRs** (touching >1 ownership area) require approval from each affected area owner.
- **Review criteria:** (1) Does it match the API contract in `types`? (2) Are there tests? (3) No secrets in code. (4) No `console.log` in production code. (5) TypeScript strict mode passes.
- **Review SLA:** Reviews should be completed within 24 hours. If not, the author pings in the team channel.
- **CI gates:** Linting (ESLint), type checking (TypeScript), unit tests (Jest), integration tests (Supertest) must all pass before merge.

13.3 Issue Tracking & Task Assignment

- **Tool:** GitHub Issues with project board (Kanban: Backlog $\rightarrow$ In Progress $\rightarrow$ In Review $\rightarrow$ Done).
- **Task granularity:** Each issue represents 1–3 days of work. Larger features are broken into sub-issues.
- **Assignment:** Issues are self-assigned during weekly planning. The ownership areas guide who picks what, but anyone can pick cross-cutting tasks.
- **Labels:** `frontend`, `backend`, `ai`, `infra`, `cross-cutting`, `bug`, `blocked`.

13.4 Communication & Decision Documentation

- **Daily:** Async standup in a dedicated Discord/Slack channel (what I did, what I'm doing, blockers).
- **Weekly:** 30-minute synchronous planning meeting to review the board and assign the next sprint's issues.
- **Decisions:** Any architectural or technical decision is documented as an ADR in `docs/decisions/` or as a comment on the relevant GitHub issue. Chat messages are ephemeral; decisions must live in the repo.

14 Development Methodology

14.1 Methodology: Scrum-Inspired Sprints (Modified)

We use a **lightweight Scrum framework** adapted for a 4-person team:

- **Sprint length:** 1 week. Short sprints force frequent integration and early detection of issues.
- **Sprint planning:** Monday, 30 minutes. Review backlog, assign issues for the week.
- **Sprint review:** Friday, 15 minutes. Demo what was completed; update stakeholders.
- **No daily standup meetings:** Replaced by async text standups (see above). A 4-person team doesn't need synchronous daily ceremonies.

14.2 Backlog Prioritization

The backlog is prioritized using a **MoSCoW** framework aligned with architectural drivers:

1. **Must Have (Sprint 1–2):** Core editing + real-time sync + basic auth + document CRUD. These are the architectural skeleton; nothing else works without them.

2. **Must Have (Sprint 3–4):** AI invocation pipeline + proposal UX + role enforcement. The product differentiator.
3. **Should Have (Sprint 5–6):** Version history, export, sharing UI, presence polish, accessibility.
4. **Could Have (Sprint 7+):** Admin dashboard, org-level settings, advanced AI features (restructure, full-document analysis).

14.3 Infrastructure & Non-Feature Work

Non-feature work (database setup, CI pipeline, testing infrastructure, Docker Compose) is explicitly scheduled in Sprint 1 as "infrastructure stories" with the same acceptance criteria rigor as feature stories. Example: "CI pipeline runs linting, type checking, and unit tests on every PR push, verified by a passing green check on a test PR." This prevents the common antipattern of perpetually deferred infrastructure.

15 Risk Assessment

Assessment Criteria

Each risk is specific to the Lidox project. For each: description, likelihood (Low/Medium/High), impact (what breaks), mitigation strategy, and contingency plan if mitigation fails.

Risk 1: CRDT Implementation Complexity Causes Integration Delays
Likelihood: Medium. *Category:* Technical.
Impact: The core real-time editing experience is blocked. Without CRDT sync, the product has no value proposition. All downstream features (AI proposals, presence, versioning) depend on a working CRDT layer.
Mitigation: Use the mature Yjs library + Hocuspocus server rather than building a custom CRDT implementation. Start with a minimal TipTap + Yjs integration in Sprint 3 (Mar 23–26) as the first real-time deliverable.
Contingency: If Yjs integration proves too complex, fall back to a simpler OT-based approach using a central server (sacrificing offline editing initially) and revisit CRDTs in a later iteration.

Risk 2: LLM API Cost Overruns During Development and Production AI-SPECIFIC
Likelihood: High. *Category:* AI Integration / Financial.
Impact: Development API costs spiral during testing. In production, a single viral document with heavy AI usage could generate hundreds of dollars in charges before detection.
Mitigation: (1) Use recorded fixtures (saved LLM responses) for all CI testing—zero real API calls in automated tests. (2) Implement per-org budget with hard-stop in Sprint 4 (Mar 27–30) *before* public deployment. (3) Set a $50/month development API budget cap with team-wide alerts.
Contingency: If costs still overrun: reduce default token ceilings, remove premium-model tier (route all tasks to fast model), or temporarily disable AI features and focus on the collaboration core.

Risk 3: WebSocket Scaling Under Load
Likelihood: Medium. *Category:* Infrastructure.
Impact: As concurrent connections grow beyond a single server's capacity (~10K connections per node), users experience connection drops, missed CRDT operations, or presence lag.
Mitigation: Design the Real-Time Service as stateless from the start: CRDT state lives in Redis, not in server memory. Multiple Real-Time Service pods can serve the same document because they all relay through Redis Pub/Sub. Load testing with k6 WebSocket plugin validates capacity before each milestone.
Contingency: If Redis Pub/Sub becomes a bottleneck: shard Pub/Sub channels by document ID range, or migrate to a dedicated message broker (NATS/Kafka) for high-throughput fan-out.

Risk 4: AI Proposal Race Condition Causes Content Corruption
Likelihood: Medium. *Category:* Technical / AI Integration.
Impact: When collaborators edit text during the 2–8 s AI round-trip, proposals anchored to stale character offsets silently corrupt document content. User trust in AI features collapses.
Mitigation: The anchor-based proposal resolution system (FR-2.2, ADR-06) detects stale proposals via CRDT state vector comparison and SHA-256 hash matching. This is implemented as a core part of Sprint 4 (AI integration, Mar 27–30), not deferred as an optimization.
Contingency: If state-vector-based anchoring proves too complex: fall back to a simpler "lock the paragraph during AI processing" approach (worse UX but prevents corruption) and iterate toward the anchor-based system in a later sprint.

Risk 5: Team Coordination Breakdown on Cross-Cutting Features TEAM-SPECIFIC

Likelihood: Medium. *Category:* Team Coordination.

Impact: The AI assistant touches frontend (proposal overlay), backend (routing), AI service (orchestration), and real-time (WebSocket delivery). If team members implement their parts with incompatible assumptions, integration fails at the end of the sprint—a classic integration hell scenario.

Mitigation: (1) Shared type definitions in `packages/types` serve as executable API contracts—if the types don't compile, the integration is broken. (2) Interface contracts are merged as the *first* PR for any cross-cutting feature, before parallel implementation begins. (3) Integration tests run in CI against all services simultaneously.

Contingency: If integration still breaks: dedicate a full "integration sprint" where all team members work together in a single call/room to wire the feature end-to-end. Reduce parallel work in favor of sequential, paired programming until the integration pattern is established.

Risk 6: LLM Provider Outage or Rate Limiting During Demo AI-SPECIFIC

Likelihood: Low–Medium. *Category:* External Dependency.

Impact: If the LLM API is down or rate-limited during a demo or grading session, the AI features appear non-functional.

Mitigation: Implement graceful degradation (NFR-07): the editing platform remains fully functional; AI toolbar shows "temporarily unavailable." Additionally, maintain a set of pre-recorded AI responses that can be replayed as a "demo mode" fallback.

Contingency: Switch to a local model (e.g., Ollama with a small open-source model) for demo purposes. Quality will be lower, but the architectural pipeline (invoke→queue→process→proposal) is demonstrated end-to-end.

16 Timeline and Milestones

Milestone Acceptance Criteria

Each milestone has concrete, verifiable acceptance criteria. "Finish backend" is not a milestone; "API serves document CRUD with authentication, verified by integration tests" is.

Sprint	Dates	Deliverables	Acceptance Criteria
Sprint 1	Mar 17–19	**M1: Architectural Skeleton.** Monorepo setup (Turborepo). Docker Compose (PG + Redis). CI pipeline (lint + type check + unit tests). Basic NestJS API scaffolding. React + TipTap editor renders locally.	(1) `turbo dev` starts all 3 apps. (2) CI passes on a test PR. (3) TipTap editor renders and accepts keystrokes locally.
Sprint 2	Mar 20–22	**M2: Core CRUD + Auth.** Document CRUD API (`POST/GET/PATCH/DELETE /api/documents`). JWT auth (login, register, refresh). Frontend connects to API: create document, load document.	(1) Integration tests pass for all document endpoints. (2) Unauthenticated requests return 401. (3) Frontend creates and loads a document via the API.
Sprint 3	Mar 23–26	**M3: Real-Time Sync.** Hocuspocus server with Yjs. WebSocket connection from frontend. Two browser tabs sync edits in real-time. Presence (cursors) displayed.	(1) Two tabs editing the same document see each other's changes within 300 ms. (2) Cursor positions are visible across tabs. (3) Disconnecting and reconnecting restores state.
Sprint 4	Mar 27–30	**M4: AI Integration.** AI invocation endpoint. Job queue (BullMQ). LLM provider adapter (one provider). Proposal UX (diff overlay, accept/reject). Token metering.	(1) User selects text and invokes "Rewrite"; AI proposal appears as a diff. (2) Accept commits to CRDT; reject dismisses. (3) Token count is logged per request.

Sprint 5	Mar 31–Apr 2	**M5: Permissions + Sharing.** Role-based access control (Owner/Editor/Commenter/Viewer). Share document by email. Permission enforcement on API and WebSocket.	(1) Viewer cannot edit. Commenter can comment but not edit. (2) Sharing by email sends invitation. (3) Permission revocation terminates active WebSocket within 60 s.
Sprint 6	Apr 3–5	**M6: Polish + Hardening.** Version history UI. Export (PDF/DOCX). Offline editing. Presence collapse ($>$10 users). Accessibility pass. Load testing.	(1) Version history shows $\geq$5 snapshots for an actively edited document. (2) Export produces valid DOCX. (3) k6 load test: 50 concurrent editors, p95 $\leq$150 ms.

Dependency Chain

M1 (skeleton) $\rightarrow$ M2 (CRUD+Auth) $\rightarrow$ M3 (real-time sync) $\rightarrow$ M4 (AI integration) $\rightarrow$ M5 (permissions) $\rightarrow$ M6 (polish). Each milestone builds on the previous. The AI integration (M4) deliberately comes *after* real-time sync (M3) because the AI proposal conflict resolution depends on a working CRDT layer.

17 Proof of Concept

PoC Deliverables

The Proof of Concept demonstrates the foundational architecture (M1 & M2 milestones): monorepo structure, strictly typed frontend-to-backend communication, configured infrastructure (Docker Compose), and the automated CI pipeline.

1. Repository Link

The complete, version-controlled source code is available here:
https://github.com/lidox-org/lidox

2. Demonstration Video

A brief video demonstrating the frontend communicating with the backend, alongside a walkthrough of the repository structure and CI/CD pipeline, can be found here: lidox-video

3. Verification Details

To verify the architecture against the repository:

- **Monorepo Structure:** Validated by the root `turbo.json` and `package.json` workspaces.
- **Data Contracts:** Validated by the shared `packages/types` folder containing Zod schemas utilized by both the NestJS API and React frontend.
- **Infrastructure:** Validated by the `docker-compose.yml` configuring PostgreSQL 16 and Redis 7 as specified in ADR-05.
- **CI/CD:** Validated by the passing GitHub Actions workflow (`.github/workflows/ci.yml`) utilizing Biome and Turborepo.

18 Glossary

Term	Definition
CRDT	Conflict-free Replicated Data Type. Concurrent updates across replicas converge without coordination.
Yjs/YATA	Yjs is a CRDT framework; YATA is its underlying sequence algorithm.
Tombstone	CRDT metadata marking a deletion. Persists until garbage-collected.
State Vector	Per-actor logical clock vector tracking causal ordering of CRDT operations.
Write-Behind	Cache pattern: writes target cache immediately, flush to DB asynchronously.
Deny Set	Redis set of revoked JWT `jti` values. Checked via O(1) `SISMEMBER`.
ProseMirror	Toolkit for rich-text editors. TipTap is built on ProseMirror.
Exponential Backoff	Retry delay doubles per attempt with random jitter.
BullMQ	Redis-based job queue for Node.js with retry and scheduling support.